Module 11: TypeScript Testing & Debugging

1. Introduction to Testing & Debugging in TypeScript

Testing ensures code correctness, while debugging helps identify and fix issues.

2. Setting Up a TypeScript Testing Environment

Install Jest for Testing

```
sh
```

npm install --save-dev jest ts-jest @types/jest

Configure Jest for TypeScript

Create `jest.config.js`:

```
js
```

module.exports = {
preset: 'ts-jest',
testEnvironment: 'node',
};

3. Writing Unit Tests with Jest

```
typescript
```

function add(a: number, b: number): number {
return a + b;
}

test("adds 2 + 3 to equal 5", () => {
expect(add(2, 3)).toBe(5);
});

4. Testing Asynchronous Code

```
typescript
```

const fetchData = async (): Promise<string> => {

return "Hello, TypeScript!";
};

test("fetchData returns correct data", async () => {
const data = await fetchData();
expect(data).toBe("Hello, TypeScript!");
});

5. Debugging TypeScript Code

Using Source Maps

Enable source maps in `tsconfig.json`:
```
json
```
{
"compilerOptions": {
"sourceMap": true
}
}

Debugging in VS Code

1. Set breakpoints in your TypeScript file.
2. Run the script with debugging:
```sh
node --inspect-brk dist/index.js
```
3. Open Chrome and visit `chrome://inspect` to debug.

6. Common TypeScript Debugging Techniques

- Use `console.log()` to trace values.
- Use `debugger;` to pause execution.
- Check for `undefined` and `null` values.

7. Error Handling in TypeScript

Using Try-Catch

```
typescript
try {
throw new Error("Something went wrong!");
} catch (error) {
console.error(error.message);
}
```

8. Best Practices for Testing & Debugging

- Write tests for critical logic first.
- Keep tests isolated and independent.
- Use debugging tools like VS Code and Chrome DevTools.
- Enable strict type checking for early error detection.

Practice Questions:

1. What is the purpose of unit testing in TypeScript?
2. How do you configure Jest for TypeScript projects?
3. Write a Jest test for a function that multiplies two numbers.
4. How does enabling source maps help in debugging?
5. Implement error handling for an API request function.

Practice Answers:

1. Unit testing ensures individual functions or components work correctly, reducing bugs in production.
2. Install Jest and configure `jest.config.js` with `preset: 'ts-jest'`.
3. Multiplication test example:

```
typescript
function multiply(a: number, b: number): number {
return a * b;
}

test("multiplies 3 * 4 to equal 12", () => {
expect(multiply(3, 4)).toBe(12);
});
```

4. Source maps allow debugging TypeScript code in the browser or VS Code by mapping compiled JavaScript back to the original TypeScript files.
5. Error handling example:

```typescript
async function fetchData(url: string): Promise<any> {
try {
const response = await fetch(url);
if (!response.ok) {
throw new Error("Network error");
}
return await response.json();
} catch (error) {
console.error("Fetch failed:", error.message);
}
}
```